

RISC-V

Single-Cycle Processor

A Complete Design & Implementation Guide in Verilog

What's Inside

- ▶ Verilog HDL essentials from scratch
- ▶ RISC-V RV32I architecture deep dive
- ▶ All 15 modules — full code analysis
- ▶ Port tables, signal flow diagrams
- ▶ Simulation & Vivado testbench guide
- ▶ Common mistakes & how to avoid them

Architecture at a Glance

IF	Instruction Fetch	PC → IMem → IR
ID	Instruction Decode	Control + RegFile + ImmGen
EX	Execute	ALU Control + ALU + Branch
MEM	Memory Access	Data Memory
WB	Write Back	WriteBack MUX → RegFile

1

Verilog Fundamentals

Everything you need before writing a single module

Verilog is a Hardware Description Language (HDL). This is fundamentally different from any programming language you have used before. You are not writing a sequence of steps for a CPU to execute — you are describing the physical structure of digital circuits: how wires connect, how logic gates combine, and how registers store values between clock edges.

The Golden Mental Model

In software: "Do step 1, then step 2, then step 3." In Verilog: "These wires are always connected like this, these flip-flops always update on this clock edge." Every line of Verilog describes hardware that exists simultaneously — not instructions that run sequentially.

1.1 The Module — *Your building block for everything*

A module is the fundamental unit in Verilog — think of it as a chip with defined input/output pins. Every component of our processor is a separate module in a separate file. The module declares what signals enter and leave it; the logic inside defines what happens to those signals.

Basic module skeleton

```
module module_name (
    input wire      clk,          // Clock — drives all synchronous logic
    input wire      reset,        // Active-high reset
    input wire [31:0] data_in,    // 32-bit input bus (bit 31 = MSB)
    output reg [31:0] data_out    // 32-bit output register
);
    // Internal logic goes here

endmodule // <-- REQUIRED: must close every module
```

- ▶ Every module begins with module <name> and must end with endmodule
- ▶ Ports are comma-separated; the last one has no trailing comma
- ▶ [31:0] declares a 32-bit vector; bit 31 is the MSB (leftmost), bit 0 is the LSB
- ▶ Best practice: one file per module, filename matches module name

1.2 Data Types — wire vs reg — *Two types with very different behaviours*

wire

Models a physical wire on a PCB. It has no memory — its value is always driven by whatever is connected to it. Disconnect the driver and it floats to high-Z.

reg

A variable used inside always blocks. The name is misleading — reg does NOT always create a hardware flip-flop:

- ✓ Left-hand side of assign
- ✓ Module output/input ports
- ✗ Cannot be left-hand side inside always

`always @(*)` → behaves like a wire (combinational)
`always @(posedge clk)` → real flip-flop, holds value

⚠ The reg Trap

A student sees "reg" and thinks "this stores data". Not always! Only a reg inside `always @(posedge clk)` creates a real D flip-flop that holds its value between clock edges. A reg inside `always @(*)` is purely combinational — the synthesiser will NOT create any storage element.

1.3 Assignment Types — Three ways to connect values

assign — Continuous Assignment (Combinational)

Use for simple combinational logic that requires no clock. The right-hand side is evaluated continuously — any change on the input immediately propagates to the output, just like a physical wire or logic gate. The left-hand side must always be a wire.

```
wire [31:0] pc_plus4;
assign pc_plus4 = pc_in + 32'd4; // Updates immediately whenever pc_in changes

wire result;
assign result = a & b;           // AND gate — pure combinational
```

= (Blocking Assignment) — Sequential within always @(*)

Used inside `always` blocks for combinational logic. Called "blocking" because each statement blocks the next — they execute in strict order, top to bottom. This models a chain of logic gates where the output of one feeds the input of the next.

```
always @(*) begin
    alu_out  = a + b;           // Step 1: compute sum
    zero_flag = (alu_out == 0); // Step 2: uses result from step 1 — correct!
end

// If you used <=, zero_flag would see the OLD value of alu_out — a bug!
```

<= (Non-Blocking Assignment) — Synchronous always @(posedge clk)

Used inside clocked `always` blocks for sequential (synchronous) logic. Called "non-blocking" because ALL right-hand sides are evaluated first (simultaneously), then ALL left-hand sides are updated together at the end of the time step. This perfectly models the behaviour of a bank of D flip-flops all sharing the same clock edge.

```
always @(posedge clk or posedge rst) begin
    if (rst) begin
        pc <= 32'b0;           // All RHS evaluated together...
        reg_a <= 32'b0;        // ...then all flip-flops updated simultaneously
    end else begin
```

```

    pc    <= pc_next; // pc gets the NEW pc_next value
    reg_a <= reg_b;    // reg_a gets the CURRENT reg_b (not the updated one)
end
end

```

❗ Iron Rule — Never Mix Assignment Types

NEVER use both = and <= in the same always block. Mixing them produces undefined, non-synthesisable behaviour that is nearly impossible to debug. Combinational always (@*) → blocking only (=). Synchronous always (@posedge clk) → non-blocking only (<=).

1.4 The always Block — How hardware is described behaviourally

The always block describes logic that "continuously runs" — re-evaluating whenever anything in its sensitivity list changes. Think of it not as a loop that runs, but as hardware that reacts to inputs.

```

// COMBINATIONAL: re-evaluates whenever any input changes
// The (*) means "sensitive to everything read inside this block"
always @(*) begin
    // Use blocking assignments (=)
    // The synthesiser creates logic gates, NOT flip-flops
end

// SYNCHRONOUS: only updates on the rising clock edge
// The posedge means "triggered by the rising edge of clk"
always @(posedge clk or posedge rst) begin
    // Use non-blocking assignments (<=)
    // The synthesiser creates D flip-flops
end

```

■ Sensitivity List Explained

The sensitivity list @(...) tells the simulator when to re-run the block. @(*) is shorthand for "every signal read inside this block". @(posedge clk) means "only on the rising edge of clk". Missing a signal from a manual sensitivity list is a classic simulation bug — always prefer @(*) for combinational logic.

2

Processor Architecture

How the 15 modules fit together

This is a single-cycle RISC-V RV32I processor. "Single-cycle" means every instruction — from a simple add to a memory load — completes in exactly one clock cycle. The clock period must be long enough for signals to propagate through the longest combinational path (the "critical path"), which for this design is typically the load-word (lw) instruction that traverses: IMem → RegFile → ALU → DMem → WB MUX → RegFile.

Stage	Modules Involved	What Happens
IF — Instruction Fetch	PC, PCplus4, branch_mux, InstructionMe	PC drives IMem; next PC selected (PC+4 or branch target)
ID — Instruction Decode	Control, RegFile, ImmGen	Opcode decoded; register values read; immediate extracted
EX — Execute	ALU_Control, ALU_Mux, ALU_unit, Branch_Addr_Calc, Branch_Logic	ALU operation performed; branch target computed
MEM — Memory Access	DataMemory	Load reads from RAM; store writes to RAM
WB — Write Back	WriteBack_Mux → RegFile	Result written back to destination register

**The Write-Back Loop**

Notice that write_back_data flows from WriteBack_Mux back into RegFile.WriteData. This is a combinational loop through purely combinational modules — it is safe because the RegFile only captures WriteData on the rising clock edge. Between clock edges, all combinational paths settle to stable values.

2.1 Instruction Formats — RV32I

Every RV32I instruction is exactly 32 bits wide. There are 6 encoding formats, each placing fields at fixed bit positions. The opcode is always in bits [6:0] — this is what our Control Unit reads.

//	31	25	24	20	19	15	14	12	11	7	6	0	
//													
//	funct7		rs2		rs1		fn3		rd		opcode		R-type (add, sub, and, or)
//													
//													
//	imm[11:0]				rs1		fn3		rd		opcode		I-type (addi, lw, jalr)
//													
//													
//	imm[11:5]		rs2		rs1		fn3		imm[4:0]		opcode		S-type (sw)
//													
//													
//	imm[20 10:1 11 19:12]										opcode		B-type (beq, bne)
//													

// Register field positions – same across ALL formats:
// rd = instruction[11:7] – destination register
// rs1 = instruction[19:15] – source register 1
// rs2 = instruction[24:20] – source register 2
// fn3 = instruction[14:12] – sub-operation code
// fn7 = instruction[31:25] – extended sub-operation code

3

Module-by-Module Analysis

Your actual code, explained in full detail

PC

Program Counter Register

File: PC.v

The Program Counter is the simplest and most critical module in the processor. It is a single 32-bit D flip-flop that holds the address of the currently executing instruction. Every single clock cycle, without exception, it captures a new value on the rising edge — either PC+4 (sequential execution) or the branch target address (taken branch).

Ports

Port Name	Direction	Width	Description
clk	input	1 bit	System clock — rising edge triggers the update
rst	input	1 bit	Active-high asynchronous reset — immediately drives PC to 0x00000000
PC_in	input	[31:0]	Next PC value to load — comes from branch_mux output
PC_out	output	[31:0]	Current PC value — drives InstructionMe and Branch_Addr_Calc

Source Code

 PC.v

```

module PC(clk, rst, PC_in, PC_out);
    input clk, rst;
    input [31:0] PC_in;
    output reg [31:0] PC_out;

    always @(posedge clk or posedge rst) begin
        if (rst)
            PC_out <= 32'b00; // Reset: jump to address 0x00000000
        else
            PC_out <= PC_in; // Normal: latch the new PC value
        end
    endmodule

```

Deep Explanation

The sensitivity list `@(posedge clk or posedge rst)` means this block fires on TWO events: the rising edge of `clk`, or the rising edge of `rst`. Because `rst` appears in the sensitivity list, the reset is asynchronous — it takes effect the instant `rst` goes high, without waiting for the next clock edge. This is essential for reliable system initialisation. The non-blocking assignment (`<=`) is mandatory here because this is a flip-flop. During the clock edge, all flip-flops in the design capture their new values simultaneously — this is the fundamental property that makes synchronous design work.

Why PC is the Only Sequential Module in the Fetch Stage

Every other module in IF (`PCplus4`, `InstructionMe`, `branch_mux`) is purely combinational. They produce their outputs continuously, right now, based on current inputs. The PC is the only one that "remembers" — it is the state element that advances the machine one instruction at a time. Remove the PC and the processor has no concept of "where it is".

PCplus4

PC Incrementer — Next Sequential Address

File: `PCplus4.v`

This is the simplest module in the design: a 32-bit adder that always adds the constant 4 to whatever PC value it receives. Adding 4 — not 1 — is because RISC-V instructions are 4 bytes (32 bits) wide, so consecutive instructions live at addresses 0, 4, 8, 12, etc. This module runs continuously and purely combinational with zero latency.

Port Name	Direction	Width	Description
<code>a_in</code>	input	[31:0]	Current PC value — connected directly to <code>PC_out</code>
<code>out</code>	output	[31:0]	<code>PC+4</code> — feeds one input of <code>branch_mux</code>

PCplus4.v

```
module PCplus4(a_in, out);
    input  [31:0] a_in;
    output [31:0] out;

    assign out = a_in + 4; // +4 bytes = next 32-bit instruction
endmodule
```

Why assign and not always?

`assign` creates a continuous, combinational connection — exactly like a logic gate. There is no clock involved, no state, no memory. The moment `a_in` changes, `out` changes. Using an `always` block here would work but would be overengineering and misleading.

InstructionMe

Instruction Memory — ROM

File: InstructionMe.v

The Instruction Memory holds the compiled program as a read-only array of 32-bit words. It contains 1024 words, giving 4KB of program storage. It operates asynchronously: as soon as a valid address arrives, the instruction appears on the output — no clock required. In a real FPGA design, this would be initialised at synthesis time or loaded via \$readmemh in simulation.

Port Name	Direction	Width	Description
rst	input	1 bit	Reset signal (not functionally used here — present for interface consistency)
A	input	[31:0]	Byte address from PC_out
InsOut	output	[31:0]	32-bit instruction at address A

▶ InstructionMe.v

```

module InstructionMe(rst, A, InsOut);
    input      rst;
    input  [31:0] A;
    output [31:0] InsOut;

    reg [31:0] mem [1023:0]; // 1024 words x 32 bits = 4 KB

    assign InsOut = mem[A[31:2]]; // Word index = byte address >> 2
endmodule

```

Addressing Explained — Why A[31:2]?

The PC keeps a byte address (the address of the first byte of the instruction). The memory array, however, stores complete 32-bit words — each array slot holds one instruction. To convert from byte address to word index, we divide by 4. In binary, dividing by 4 means dropping the two least-significant bits:

```

// Byte address 0x00000000 → mem[0]   (0000 0000 >> 2 = 0)
// Byte address 0x00000004 → mem[1]   (0000 0100 >> 2 = 1)
// Byte address 0x00000008 → mem[2]   (0000 1000 >> 2 = 2)
// Byte address 0x000000FC → mem[63]  (1111 1100 >> 2 = 63)

```

```

// The two dropped bits (A[1:0]) are always 00 in a well-aligned program.
// If PC is ever not word-aligned, those bits are silently ignored.

```

Loading the Program in Simulation

In your testbench, use: `$readmemh("program.hex", dut.imem.mem);` The .hex file contains the machine-code instructions in hexadecimal, one per line. Each line loads one 32-bit word into the memory array.

Control

Main Control Unit — The Decoder

File: Control.v

The Control Unit is the "brain" of the processor. It reads only the 7-bit opcode field from the instruction and generates all the control signals that steer every other module for that instruction type. It is a purely combinational, stateless block — a large truth table implemented as a case statement.

Port Name	Direction	Width	Description
Opcode	input	[6:0]	instruction[6:0] — the operation type field
Branch	output	1 bit	1 = this is a branch instruction (feeds Branch_Logic)
MemRead	output	1 bit	1 = read data memory this cycle (lw only)
MemtoReg	output	1 bit	0 = write ALU result to reg 1 = write memory data to reg
ALUOp	output	[1:0]	Coarse ALU operation hint sent to ALU_Control
MemWrite	output	1 bit	1 = write data memory this cycle (sw only)
ALUSrc	output	1 bit	0 = ALU B-input from rs2 1 = ALU B-input from immediate
RegWrite	output	1 bit	1 = write result back to register file this cycle

Control.v

```

module Control(Opcode, Branch, MemRead, MemtoReg,
               ALUOp, MemWrite, ALUSrc, RegWrite);
    input [6:0] Opcode;
    output reg Branch, MemRead, MemtoReg;
    output reg [1:0] ALUOp;
    output reg MemWrite, ALUSrc, RegWrite;

    always @(*) begin
        // Default all signals to 0 - safe fallback for unknown opcodes
        {Branch, MemRead, MemtoReg, MemWrite, ALUSrc, RegWrite} = 6'b0;
        ALUOp = 2'b00;

        case (Opcode)
            7'b0110011: begin // R-type: add, sub, and, or, xor, slt
                RegWrite = 1; // Write result to rd
                ALUSrc = 0; // ALU B = rs2 (register operand)
                MemtoReg = 0; // Write ALU result (not memory)
                ALUOp = 2'b10; // ALU_Control will decode funct3/7
            end
        end
    end

```

```

7'b0000011: begin // I-type Load: lw
    RegWrite = 1; // Write loaded value to rd
    ALUSrc   = 1; // ALU B = sign-extended immediate (offset)
    MemtoReg = 1; // Write MEMORY data to register
    MemRead  = 1; // Enable data memory read port
    ALUOp    = 2'b00; // ALU always adds: base + offset
end

7'b0100011: begin // S-type Store: sw
    MemWrite = 1; // Write to data memory
    ALUSrc   = 1; // ALU B = immediate (store offset)
    ALUOp    = 2'b00; // ALU adds base + offset = address
end

7'b1100011: begin // B-type Branch: beq
    Branch   = 1; // Signal Branch Logic to check condition
    ALUSrc   = 0; // ALU B = rs2 (for subtraction rs1 - rs2)
    ALUOp    = 2'b01; // ALU subtracts; zero flag = rs1==rs2
end

7'b0010011: begin // I-type ALU-Immediate: addi
    RegWrite = 1;
    ALUSrc   = 1; // ALU B = immediate
    ALUOp    = 2'b00; // Note: addi always adds
end

default: ; // All signals remain 0 – NOP effect
endcase
end
endmodule

```

Control Signal Truth Table

Instruction	Opcode	RegWrite	ALUSrc	MemRead	MemWrite	MemtoReg	Branch	ALUOp
R-type (add/sub...)	0110011	1	0	0	0	0	0	10
lw (load word)	0000011	1	1	1	0	1	0	00
sw (store word)	0100011	0	1	0	1	x	0	00
beq (branch eq)	1100011	0	0	0	0	x	1	01
addi (add imm)	0010011	1	1	0	0	0	0	00

Why Default to All Zeros?

Setting all control outputs to 0 at the start of the always block (before the case) is a safety pattern. If an unrecognised opcode arrives, nothing dangerous happens: no memory write, no register write, no branch.

Without this, a missing case arm would create latches in synthesis (because the reg would need to "remember" its previous value).

RegFile

Register File — 32×32-bit Registers

File: RegFile.v

The register file is the processor's fast, on-chip storage. It holds 32 general-purpose 32-bit registers (x0–x31). It provides two simultaneous asynchronous read ports (rs1 and rs2 can be read in the same cycle) and one synchronous write port (the result of the previous instruction written on the rising clock edge). This is a classic multi-ported SRAM structure.

Port Name	Direction	Width	Description
clk	input	1 bit	Clock — write occurs on rising edge
rst	input	1 bit	Reset — clears all 32 registers to 0x00000000
RegWrite	input	1 bit	Write enable from Control Unit
readReg1	input	[4:0]	rs1 address — instruction[19:15]
readReg2	input	[4:0]	rs2 address — instruction[24:20]
WriteRegister	input	[4:0]	rd address — instruction[11:7]
WriteData	input	[31:0]	Data to write — from WriteBack_Mux
ReadData1	output	[31:0]	Value of rs1 — feeds ALU operand A
ReadData2	output	[31:0]	Value of rs2 — feeds ALU_Mux and DataMemory.writeData

RegFile.v

```

module RegFile(clk, rst, RegWrite,
               readReg1, readReg2, WriteRegister,
               WriteData, ReadData1, ReadData2);

    integer k;
    reg [31:0] registers [31:0];    // Array: 32 registers, each 32 bits wide

    // WRITE PORT — synchronous, clock-driven
    always @(posedge clk or posedge rst) begin
        if (rst)
            for(k = 0; k < 32; k = k + 1)
                registers[k] <= 32'b0;    // Clear all registers on reset
        else if (RegWrite && (WriteRegister != 5'b0))
            registers[WriteRegister] <= WriteData;    // Write on clock edge
    end

    // READ PORTS — asynchronous, instant output (no clock needed)
    // x0 is hardwired to zero — reading it always returns 0

```

```

assign ReadData1 = (readReg1 == 5'b0) ? 32'b0 : registers[readReg1];
assign ReadData2 = (readReg2 == 5'b0) ? 32'b0 : registers[readReg2];

endmodule

```

Why is x0 Hardwired to Zero?

RISC-V mandates that register x0 always reads as 0 and ignores all writes. This gives you a free zero constant without using an instruction. It enables useful pseudo-instructions: "mv rd, rs" = "add rd, rs, x0" | "nop" = "addi x0, x0, 0" | "neg rd, rs" = "sub rd, x0, rs". The check (WriteRegister != 5'b0) in the write port enforces this — any attempt to write x0 is silently discarded.

ImmGen

Immediate Generator — Sign Extender

File: ImmGen.v

The Immediate Generator extracts the embedded constant (immediate) from the instruction encoding and sign-extends it to 32 bits. "Sign extension" means replicating the sign bit (bit 31 of the instruction) into all upper bits, preserving the value and sign of the constant. Each instruction format scatters the immediate bits to different positions — ImmGen reassembles them in the correct order.

Port Name	Direction	Width	Description
instruction	input	[31:0]	Full 32-bit instruction from InstructionMe
ImmExt	output	[31:0]	Sign-extended 32-bit immediate value

ImmGen.v

```

module ImmGen(instruction, ImmExt);
    input  [31:0] instruction;
    output reg [31:0] ImmExt;

    wire [6:0] opcode = instruction[6:0]; // Extract opcode for format detection

    always @(*) begin
        case (opcode)
            // I-type: lw, addi, etc.
            // Immediate is bits [31:20] — 12 bits, sign-extended to 32
            7'b0000011, // LOAD
            7'b0010011: // OP-IMM
                ImmExt = {{20{instruction[31]}}, instruction[31:20]};
                //      ^^^^^^^^^^^^^^^^^^^^^^^^^ replicate sign bit 20 times

            // S-type: sw
            // Immediate is SPLIT: bits [31:25] are upper, [11:7] are lower

```

```

// Why split? To keep rs1/rs2 at the same positions as R-type!
7'b0100011:
    ImmExt = {{20{instruction[31]}},
               instruction[31:25], // imm[11:5]
               instruction[11:7]}; // imm[4:0]

// B-type: beq, bne
// Immediate encodes a SIGNED BYTE OFFSET to the branch target
// Bits are scrambled across the instruction – reassembled here
// Bit 0 is always 0 (branches are word-aligned)
7'b1100011:
    ImmExt = {{19{instruction[31]}}, // sign extension
               instruction[31],      // imm[12]
               instruction[7],       // imm[11]
               instruction[30:25],   // imm[10:5]
               instruction[11:8],    // imm[4:1]
               1'b0};               // imm[0] = 0 always

default: ImmExt = 32'b0;
endcase
end
endmodule

```

Sign Extension Deep Dive

The Verilog replication syntax `{N{expr}}` repeats the expression `N` times. So `{20{instruction[31]}}` creates 20 copies of the sign bit. If the immediate is positive (bit 31 = 0), all 20 upper bits are 0. If negative (bit 31 = 1), all 20 upper bits are 1. This preserves the two's complement value.

```

// Example: addi x1, x0, -5
// In I-type, immediate field [31:20] = 1111_1111_1011 (12-bit two's complement of -5)
// instruction[31] = 1 (negative)
// After ImmGen: {{20{1}}, 1111_1111_1011} = 0xFFFFFFF B = -5 in 32-bit two's complement ✓

// Example: addi x1, x0, 7
// Immediate field [31:20] = 0000_0000_0111
// instruction[31] = 0 (positive)
// After ImmGen: {{20{0}}, 0000_0000_0111} = 0x00000007 = +7 ✓

```

B-type Bit Scrambling

The B-type immediate has its bits scattered non-sequentially through the instruction. This is intentional in the RISC-V spec — it minimises the hardware needed to decode multiple formats by keeping most bits at the same positions. The key points: `imm[12] = instruction[31]`, `imm[11] = instruction[7]`, `imm[10:5] = instruction[30:25]`, `imm[4:1] = instruction[11:8]`, `imm[0] = 0`. Always bit 0 is 0 because branches target word-aligned addresses.

ALU_Control

ALU Control Unit — Operation Decoder

File: ALU_Control.v

The Control Unit produces a coarse 2-bit ALUOp hint. The ALU_Control module refines this into a precise 4-bit operation code by also examining funct7[30] and funct3[2:0] from the instruction. This two-level decoding reduces the complexity of the main Control Unit and keeps concerns separated.

Port Name	Direction	Width	Description
ALUOp	input	[1:0]	00 = add always 01 = subtract always 10 = decode from funct
fun7	input	1 bit	instruction[30] — distinguishes ADD(0) from SUB(1)
fun3	input	[2:0]	instruction[14:12] — encodes AND/OR/XOR/SLT etc.
Control_out	output	[3:0]	4-bit operation code sent to ALU_unit

ALU_Control.v

```

module ALU_Control(ALUOp, fun7, fun3, Control_out);
    input [1:0] ALUOp;
    input      fun7;
    input [2:0] fun3;
    output reg [3:0] Control_out;

    always @(*) begin
        Control_out = 4'b0010; // Safe default: ADD

        // Concatenate all inputs into a 6-bit lookup key
        case({ALUOp, fun7, fun3})
            6'b00_0_000: Control_out = 4'b0010; // lw/sw: ADD (base + offset)
            6'b01_0_000: Control_out = 4'b0110; // beq: SUB (rs1 - rs2, check zero)
            6'b10_0_000: Control_out = 4'b0010; // R-type: add
            6'b10_1_000: Control_out = 4'b0110; // R-type: sub (funct7[30]=1)
            6'b10_0_111: Control_out = 4'b0000; // R-type: and
            6'b10_0_110: Control_out = 4'b0001; // R-type: or
            default:      Control_out = 4'b0010; // Fallback: ADD
        endcase
    end
endmodule

```

ALU Encoding Table

Instruction	ALUOp	funct7[30]	funct3	Control_out	ALU Operation
lw, sw (address)	00	x	xxx	0010	ADD (base + offset)

beq (compare)	01	x	xxx	0110	SUB (rs1 - rs2, test zero)
add	10	0	000	0010	ADD
sub	10	1	000	0110	SUB
and	10	0	111	0000	AND
or	10	0	110	0001	OR



The Two-Level Decode Pattern

The main Control Unit only needs to know "is this R-type?" (ALUOp=10), "is this always add?" (ALUOp=00), or "is this always subtract?" (ALUOp=01). ALU_Control then handles the fine-grained decoding for R-type instructions using funct7 and funct3. This pattern keeps each module small, testable, and easy to extend.

ALU_Mux

ALU Input Multiplexer — B-Operand Selector

File: ALU_Mux.v

A simple 2-to-1 multiplexer that selects which value becomes the B operand (second input) of the ALU. For R-type instructions, B is a register value (rs2). For I-type and S-type instructions, B is the sign-extended immediate. The ALUSrc control signal from the Control Unit makes this choice.

Port Name	Direction	Width	Description
ReadData2	input	[31:0]	rs2 value from register file — used by R-type instructions
ImmExt	input	[31:0]	Sign-extended immediate from ImmGen — used by I-type, S-type
ALUSrc	input	1 bit	0 = select ReadData2 1 = select ImmExt
out	output	[31:0]	ALU B operand — wired to ALU_unit input B

ALU_Mux.v

```

module ALU_Mux(ReadData2, ImmExt, ALUSrc, out);
    input [31:0] ReadData2, ImmExt;
    input ALUSrc;
    output [31:0] out;

    // Ternary operator: if ALUSrc==1 then ImmExt, else ReadData2
    assign out = (ALUSrc == 1'b1) ? ImmExt : ReadData2;
endmodule

```

Example walkthrough: add x1, x2, x3 → ALUSrc=0 → out = ReadData2 = value of x3. addi x1, x2, 10 → ALUSrc=1 → out = ImmExt = 32'd10.

ALU_unit

Arithmetic Logic Unit — The Calculator

File: ALU_unit.v

The ALU is the heart of the execute stage. It receives two 32-bit operands and performs the arithmetic or logical operation specified by the 4-bit control code. It also produces a zero flag — a 1-bit output that goes high when the result is exactly zero, which is used by the branch logic to decide whether a conditional branch should be taken.

Port Name	Direction	Width	Description
A	input	[31:0]	First operand — always from ReadData1 (rs1 value)
B	input	[31:0]	Second operand — from ALU_Mux (rs2 or immediate)
ALU_Control	input	[3:0]	4-bit operation code from ALU_Control module
res	output	[31:0]	Operation result — goes to DataMemory.address and WriteBack_Mux
zero	output	1 bit	HIGH when res == 0 — used to trigger conditional branches

ALU_unit.v

```

module ALU_unit(A, B, ALU_Control, res, zero);
    input [31:0] A, B;
    input [3:0] ALU_Control;
    output reg [31:0] res;
    output reg zero;

    always @(*) begin
        zero = 1'b0; // Default zero to 0; computed below

        case(ALU_Control)
            4'b0000: res = A & B; // AND
            4'b0001: res = A | B; // OR
            4'b0010: res = A + B; // ADD
            4'b0110: res = A - B; // SUB (also used by beq to test equality)
            default: res = 32'b0; // Safe fallback
        endcase

        zero = (res == 32'b0); // Computed AFTER case — reflects actual result
    end
endmodule

```

How beq Uses the ALU

For beq rs1, rs2, offset: ALU_Control receives 0110 (SUB), so res = rs1 – rs2. If rs1 == rs2, then res = 0, so zero = 1. The zero flag travels to Branch_Logic. Branch_Logic checks: Branch (from Control) AND zero (from ALU). Only

when BOTH are 1 does `pc_src` become 1, causing the branch to be taken. This is clever: the ALU does equality checking for free using subtraction.

Branch_Addr_Calc

Branch Target Address Calculator

File: `Branch_Addr_Calc.v`

Computes the target address that the PC should jump to if a branch is taken. The formula is simple: `target = PC_current + immediate`. The immediate for B-type instructions encodes a signed byte offset — it can be positive (forward branch) or negative (backward branch, e.g., loop). This module runs in parallel with the ALU — both compute during the EX stage simultaneously.

Port Name	Direction	Width	Description
<code>pc_in</code>	input	[31:0]	Current PC value — the base address for the branch
<code>imm_ext</code>	input	[31:0]	Sign-extended B-type immediate from ImmGen (the offset)
<code>br_addr_out</code>	output	[31:0]	Branch target address — feeds one input of <code>branch_mux</code>

Branch_Addr_Calc.v

```

module Branch_Addr_Calc(pc_in, imm_ext, br_addr_out);
    input  [31:0] pc_in;
    input  [31:0] imm_ext;
    output [31:0] br_addr_out;

    assign br_addr_out = pc_in + imm_ext;
    // Example: beq at PC=0x10, with imm_ext=0x8 → target = 0x18
    // Example: beq at PC=0x40, with imm_ext=0xFFFFFFC (-4) → target = 0x3C
endmodule

```

Branch_Logic

Branch Decision Unit

File: `Branch_Logic.v`

This is the decision gate that determines whether a conditional branch is actually taken. It combines two inputs with a single AND gate. Both conditions must be true simultaneously: (1) the current instruction IS a branch instruction (`Branch=1` from Control Unit), and (2) the branch condition IS satisfied (`alu_zero=1`, meaning `rs1==rs2` for `beq`).

Port Name	Direction	Width	Description
-----------	-----------	-------	-------------

branch_ctrl	input	1 bit	Branch signal from Control Unit — 1 if instruction is a branch
alu_zero	input	1 bit	Zero flag from ALU — 1 if $(rs1 - rs2 == 0) \rightarrow rs1 == rs2$
pc_src	output	1 bit	0 = take PC+4 1 = take branch target address

▢ Branch_Logic.v

```

module Branch_Logic(branch_ctrl, alu_zero, pc_src);
    input  branch_ctrl;
    input  alu_zero;
    output pc_src;

    // Branch ONLY if: this IS a branch instruction AND the condition holds
    assign pc_src = branch_ctrl & alu_zero;
endmodule

```

⚠ What Happens Without the AND?

If you wired only `alu_zero` to `pc_src` (forgetting `branch_ctrl`), then every instruction that produces a zero result (e.g., `add x1, x0, x0`) would cause a spurious branch! Both conditions are essential. Similarly, if you wired only `branch_ctrl`, the branch would always be taken whenever the instruction is `beq` — regardless of whether `rs1` actually equals `rs2`.

branch_mux

PC Source Multiplexer — Next PC Selector

File: `branch_mux.v`

This final multiplexer in the instruction fetch path selects what value gets loaded into the PC on the next clock edge. In the common case (no branch taken), it passes PC+4 (sequential execution). When a branch is taken (`pc_src=1` from `Branch_Logic`), it passes the branch target address instead. Its output feeds directly into `PC.PC_in`.

Port Name	Direction	Width	Description
pc_plus_4	input	[31:0]	PC+4 from PCplus4 — sequential next address
pc_branch_target	input	[31:0]	Branch target from Branch_Addr_Calc
pcSrc	input	1 bit	Select signal from Branch_Logic — 0=sequential, 1=branch
out	output	[31:0]	Next PC value — wired to PC.PC_in

▢ branch_mux.v

```

module branch_mux(pc_plus_4, pc_branch_target, pcSrc, out);
    input [31:0] pc_plus_4, pc_branch_target;
    input  pcSrc;

```

```
output [31:0] out;
```

```
assign out = (pcSrc == 1'b1) pc_branch_target : pc_plus_4;
// pcSrc=0: out = pc_plus_4      (most instructions)
// pcSrc=1: out = pc_branch_target (taken branch)
```

```
endmodule
```

DataMemory

Data Memory — RAM for lw/sw Instructions

File: DataMemory.v

Data Memory provides byte-addressable read/write storage for the program's variables and stack. This implementation has 64 words (256 bytes) of RAM, enough for simple programs. Reads are asynchronous — data appears immediately when a valid address and memRead=1 are applied. Writes are synchronous — data is stored on the rising clock edge when memWrite=1. This asymmetry is intentional and standard: it allows reads in the MEM stage without adding a cycle.

Port Name	Direction	Width	Description
clk	input	1 bit	Clock — write happens on rising edge
rst	input	1 bit	Reset — zeros all 64 memory locations
memWrite	input	1 bit	Write enable — from Control Unit (sw only)
memRead	input	1 bit	Read enable gate — from Control Unit (lw only)
address	input	[31:0]	Memory address — comes from ALU result (base + offset)
writeData	input	[31:0]	Data to store — from register file ReadData2 (rs2 value)
readData	output	[31:0]	Data read — goes to WriteBack_Mux for lw

▶ DataMemory.v

```
module DataMemory(clk, rst, memWrite, memRead,
                  address, writeData, readData);
    reg [31:0] RAM [0:63]; // 64 words × 32 bits = 256 bytes of RAM
    integer i;

    // ASYNCHRONOUS READ — data available immediately
    // If memRead=0, output 0 (prevents stale data from appearing)
    assign readData = (memRead) RAM[address[7:2]] : 32'b0;

    // SYNCHRONOUS WRITE + RESET
    always @(posedge clk or posedge rst) begin
        if (rst)
            for (i = 0; i < 64; i = i + 1)
                RAM[i] <= 32'b0; // Clear memory on reset
        else if (memWrite)
            RAM[address[7:2]] <= writeData; // Store word at word-aligned address
    end
endmodule
```

```
end
endmodule
```

Address Alignment — address[7:2]

Like InstructionMe, DataMemory stores 32-bit words but receives byte addresses from the ALU. address[7:2] performs the divide-by-4: it takes bits 7 down to 2, which is equivalent to a logical right shift by 2 positions. The two discarded bits [1:0] should always be 00 for properly aligned word accesses (lw/sw with word-aligned addresses). Misaligned accesses would silently truncate.

```
// lw x5, 0(x1) — x1 = 0x00000010 (address 16)
// ALU result = 0x00000010
// address[7:2] = 0b000100 = 4 → RAM[4]

// sw x6, 8(x1) — x1 = 0x00000010 (address 16)
// ALU result = 0x00000018 (16 + 8 = 24)
// address[7:2] = 0b000110 = 6 → RAM[6] ← value of x6
```

WriteBack_Mux

Write-Back Multiplexer — Result Selector

File: WriteBack_Mux.v

The final decision-maker before data enters the register file: for most instructions, the result to write back is the ALU's output. For a load word (lw), the value comes from data memory. MemtoReg from the Control Unit makes this selection. For instructions that don't write back (sw, beq), RegWrite=0 prevents any update regardless of what this MUX outputs.

Port Name	Direction	Width	Description
ALU_Result	input	[31:0]	Result from ALU_unit — for R-type, I-ALU, branch address computation
Mem_Data	input	[31:0]	Data read from DataMemory — only valid for lw instructions
MemtoReg	input	1 bit	0 = write ALU result 1 = write memory data (lw)
out	output	[31:0]	WriteData input to RegFile — the value written to rd

WriteBack_Mux.v

```
module WriteBack_Mux(ALU_Result, Mem_Data, MemtoReg, out);
    input [31:0] ALU_Result, Mem_Data;
    input MemtoReg;
    output [31:0] out;
```

```

assign out = (MemtoReg == 1'b1)  Mem_Data : ALU_Result;
// MemtoReg=0: add, sub, and, or, addi → write ALU result
// MemtoReg=1: lw → write data loaded from memory
endmodule

```

Risc_V_top_module

Top-Level Module — The Complete Processor

File: Risc_V_top_module.v

The top module instantiates and wires together all 14 sub-modules. It declares the internal wires that carry signals between modules, and exposes the most important signals as outputs for simulation observability (connecting to the testbench's waveform viewer).

Port Name	Direction	Width	Description
clk	input	1 bit	System clock — shared by all synchronous modules
rst	input	1 bit	Global reset — propagates to PC, RegFile, DataMemory
pc_out	output	[31:0]	Current program counter (observable)
instruction	output	[31:0]	Current instruction being executed (observable)
read_data1	output	[31:0]	rs1 register value (observable)
read_data2	output	[31:0]	rs2 register value (observable)
imm_ext	output	[31:0]	Sign-extended immediate (observable)
reg_write_ctrl	output	1 bit	RegWrite control signal (observable)
alu_op_ctrl	output	[1:0]	ALUOp control signal (observable)
alu_result	output	[31:0]	ALU output — address or arithmetic result (observable)
write_back_data	output	[31:0]	Value being written to register file (observable)

Internal Wire Declarations

```

// All internal connections between modules
wire [31:0] pc_next;           // Selected next PC → PC.PC_in
wire [31:0] pc_plus_4;        // PC+4 → branch_mux input 0
wire [31:0] pc_branch_target; // PC+offset → branch_mux input 1
wire [31:0] alu_input_b;      // ALU B operand (rs2 or immediate)
wire [31:0] mem_read_data;    // Data loaded from DataMemory
wire [3:0] alu_control_signal; // Fine-grained ALU operation code
wire      alu_src;            // MUX select: reg vs immediate
wire      mem_to_reg;         // MUX select: ALU vs memory
wire      mem_read;          // DataMemory read enable
wire      mem_write;         // DataMemory write enable
wire      branch;            // This instruction is a branch
wire      zero_flag;         // ALU result is zero

```

```
wire      pc_sel;           // 0=PC+4, 1=branch target
```

Complete Module Instantiations

▢ Risc_V_top_module.v (complete)

```
// — Stage IF: Instruction Fetch —
PC pc_inst (
    .clk(clk), .rst(rst),
    .PC_in(pc_next), // Receives selected next PC from branch_mux
    .PC_out(pc_out)  // Drives everything that needs current address
);

PCplus4 pc_plus (
    .a_in(pc_out),
    .out(pc_plus_4) // PC+4 → one input of branch_mux
);

branch_mux muxNextPc (
    .pc_plus_4(pc_plus_4),
    .pc_branch_target(pc_branch_target),
    .pcSrc(pc_sel),
    .out(pc_next) // → PC_in: closes the PC feedback loop
);

InstructionMe imem (
    .rst(rst), .A(pc_out),
    .InsOut(instruction) // 32-bit instruction drives everything below
);

// — Stage ID: Instruction Decode —
Control control_unit (
    .Opcode(instruction[6:0]), // Bottom 7 bits of instruction
    .Branch(branch), .MemRead(mem_read), .MemtoReg(mem_to_reg),
    .ALUOp(alu_op_ctrl), .MemWrite(mem_write),
    .ALUSrc(alu_src), .RegWrite(reg_write_ctrl)
);

RegFile registers (
    .clk(clk), .rst(rst), .RegWrite(reg_write_ctrl),
    .readReg1(instruction[19:15]), // rs1 field
    .readReg2(instruction[24:20]), // rs2 field
    .WriteRegister(instruction[11:7]), // rd field
    .WriteData(write_back_data), // From WB stage (feedback path)
    .ReadData1(read_data1), .ReadData2(read_data2)
);

ImmGen imm_gen_unit (
    .instruction(instruction),
    .ImmExt(imm_ext)
);
```

```

// — Stage EX: Execute —
Branch_Addr_Calc br_adder (
    .pc_in(pc_out),
    .imm_ext(imm_ext),          // B-type immediate
    .br_addr_out(pc_branch_target)
);

Branch_Logic br_logic (
    .branch_ctrl(branch),      // From Control Unit
    .alu_zero(zero_flag),      // From ALU
    .pc_src(pc_sel)            // → branch_mux_select
);

ALU_Mux alu_mux_inst (
    .ReadData2(read_data2),
    .ImmExt(imm_ext),
    .ALUSrc(alu_src),
    .out(alu_input_b)          // → ALU B input
);

ALU_Control alu_ctrl_unit (
    .ALUOp(alu_op_ctrl),
    .fun7(instruction[30]),     // Single bit: distinguishes add/sub
    .fun3(instruction[14:12]), // 3 bits: operation type
    .Control_out(alu_control_signal)
);

ALU_unit alu_inst (
    .A(read_data1),
    .B(alu_input_b),
    .ALU_Control(alu_control_signal),
    .res(alu_result),
    .zero(zero_flag)
);

// — Stage MEM: Memory —
DataMemory dmem (
    .clk(clk), .rst(rst),
    .memWrite(mem_write), .memRead(mem_read),
    .address(alu_result),      // ALU computed the address
    .writeData(read_data2),    // rs2 value to store (for sw)
    .readData(mem_read_data)
);

// — Stage WB: Write Back —
WriteBack_Mux wb_mux (
    .ALU_Result(alu_result),
    .Mem_Data(mem_read_data),
    .MemtoReg(mem_to_reg),
    .out(write_back_data)      // → RegFile.WriteData (closes the loop)
);

```


4

Signal Flow Trace

Follow each instruction type through the datapath

Understanding how control signals and data flow through the processor for each instruction type is key to debugging. The table below traces what happens at each stage for the four main instruction types.

Stage	R-type (add x1,x2,x3)	lw x1, 8(x2)	sw x3, 8(x2)	beq x1,x2, offset
IF	PC→IMem→instruction	PC→IMem→instruction	PC→IMem→instruction	PC→IMem→instruction
ID	RegFile reads x2, x3 Control: ALUSrc=0, ALUOp=10	RegFile reads x2 ImmGen→8 Control: ALUSrc=1, MemRead=1	RegFile reads x2,x3 ImmGen→8 Control: ALUSrc=1, MemWrite=1	RegFile reads x1,x2 Control: Branch=1, ALUOp=01
EX	ALU_Mux→ReadData2 ALU: $x2 + x3 =$ result	ALU_Mux→ImmExt(8) ALU: $x2 + 8 =$ address	ALU_Mux→ImmExt(8) ALU: $x2 + 8 =$ address	ALU: $x1 - x2$ BrAddrCalc: PC+offset zero=(x1==x2)
MEM	Not used	RAM[address] → readData	RAM[address] ← x3	pc_src = Branch & zero Next PC selected
WB	$x1 \leftarrow$ ALU result (MemtoReg=0, RegWrite=1)	$x1 \leftarrow$ readData (MemtoReg=1, RegWrite=1)	No write-back (RegWrite=0)	No write-back (RegWrite=0)

Critical Path Analysis

The "critical path" is the longest combinational delay through the processor — it determines the minimum clock period (maximum clock frequency). For this design, the critical path runs through the lw instruction:

// Critical path for lw (the slowest instruction):
//
// PC_out
// → InstructionMe (ROM read delay)
// → ImmGen (immediate extraction)
// → ALU_Mux (MUX delay)
// → ALU_unit (32-bit addition)
// → DataMemory (RAM read delay)
// → WriteBack_Mux (MUX delay)
// → RegFile.WriteData (setup time for flip-flop)
//
// Clock period must be $\geq$ sum of all these propagation delays.
// In a real 28nm process this might be ~1-2ns → 500MHz-1GHz.
// On a typical student FPGA (Artix-7), this limits you to ~50-100MHz.

5

Simulation & Testbench

Verifying your processor in Vivado

A testbench is a special Verilog module that exists only for simulation — it is never synthesised into hardware. It instantiates your processor (the "Device Under Test"), generates clock and reset signals, loads a test program into instruction memory, and observes outputs to verify correct behaviour.

▶ Complete testbench example

```
`timescale 1ns / 1ps // Time unit = 1ns, precision = 1ps

module tb_Risc_V_top;
    // Testbench drives these as reg (we assign them in initial/always blocks)
    reg clk;
    reg rst;

    // Instantiate the processor (DUT = Device Under Test)
    Risc_V_top_module dut (
        .clk(clk),
        .rst(rst)
        // All other ports are outputs – connect if you want to monitor them
    );

    // — Generate a 10ns clock (100 MHz) —————
    initial clk = 1'b0; // Start low
    always #5 clk = ~clk; // Toggle every 5ns → period = 10ns

    // — Load program and run —————
    initial begin
        // Load machine code into instruction memory
        // program.hex contains hex words, one per line
        $readmemh("program.hex", dut.imem.mem);

        // Apply reset for 2 full clock cycles
        rst = 1'b1;
        #20; // Hold reset for 20ns (2 cycles)
        rst = 1'b0; // Release reset – processor starts executing

        // Run for 1000 cycles then stop
        #1000;
        $display("=== Simulation complete ===");
        $finish;
    end

    // — Monitor – print one line per clock cycle —————
    always @(posedge clk) begin
        $display("t=%0t PC=0x%h INSTR=0x%h WB=0x%h",
            $time,
            dut.pc_out,
            dut.instruction,
            dut.write_back_data);
    end
end
```

```
endmodule
```

Running in Vivado — Step by Step

1. Create a new project: File → Project → New → RTL Project
2. Add all 15 .v files as Design Sources (Add Sources → Add or create design sources)
3. Add your testbench .v file as a Simulation Source (Add Sources → Simulation sources)
4. Ensure your testbench is set as the top simulation module
5. Run: Flow Navigator → Simulation → Run Simulation → Run Behavioral Simulation
6. In the waveform viewer, drag signals from the Scope panel to the wave window
7. Click "Run All" to simulate. Examine the waveforms at each rising clock edge

What to Check in the Waveforms

```
// 1. PC increments by 4 each cycle (when no branch)
//    PC:  0x00 → 0x04 → 0x08 → 0x0C → ...

// 2. After a taken beq, PC jumps to the branch target:
//    PC:  0x10 → 0x18  (jumped over one instruction)

// 3. After lw x5, 0(x1) – check write_back_data on the NEXT cycle:
//    It should show the value that was in RAM at the loaded address.

// 4. After add x3, x1, x2 – check write_back_data:
//    Should equal (value of x1) + (value of x2)

// 5. Confirm alu_result is 0 when beq compares equal registers.
//    This should trigger pc_sel = 1 on the same cycle.
```



Debugging Checklist

- PC stuck at 0? → Check rst is deasserted; check that branch_mux output reaches PC_in.
- All instructions look like 0xFFFFFFFF? → Memory not loaded. Check \$readmemh path.
- Register never updates? → Check RegWrite=1 and WriteRegister is correct.
- Branch never taken? → Check zero_flag in waveform AND that Branch=1 from Control.
- lw returns garbage? → Check address[7:2] alignment and MemRead=1.

6

Common Mistakes & How to Fix Them

The bugs that trip up every beginner **Mixing = and <= in the same always block**

Symptom: simulation works but synthesis produces wrong behaviour, or simulator gives warnings. Fix: combinational always @(*) → use = exclusively. Synchronous always @(posedge clk) → use <= exclusively. No exceptions.

 Using wire on the left-hand side of always

Symptom: "cannot drive wire from procedural block" error. Fix: any signal that is assigned inside an always block must be declared as reg. Use wire only for continuous assign and module port connections.

 Forgetting default in a case statement

Symptom: simulation runs but synthesis creates latches (memory elements you did not intend). Fix: always add a default: arm. For the Control Unit, the default is to set everything to 0. Latches are inferred when a reg can be reached without being assigned — default prevents this.

 Writing to x0 in the register file

Symptom: hard-to-trace bugs when x0 is expected to be 0 but contains garbage. Fix: always guard the write port: if (RegWrite && WriteRegister != 5'b0). The RISC-V spec mandates x0 is always 0 — enforcing this in hardware is your responsibility.

 Incorrect B-type immediate assembly in ImmGen

Symptom: beq jumps to a wrong address. The immediate is close but not quite right. Fix: the B-type encoding is instruction[31]=imm[12], instruction[7]=imm[11], instruction[30:25]=imm[10:5], instruction[11:8]=imm[4:1], and imm[0] is always 0.

 Missing endmodule

Symptom: confusing parse errors on the next module in the file. Verilog merges them. Fix: every module begins with module and ends with endmodule. One module per file makes this easier to catch.

 Branch never taken even when condition holds

Symptom: beq loop runs forever. Fix: check both inputs to Branch_Logical. If Branch (from Control) is 0, no branch fires. If ALU_Control is not 0110 (SUB) for beq, the zero flag will not reflect equality.

 Width mismatches silently truncate

Symptom: a 32-bit value arrives as 28 bits on the other side; upper bits are missing. Fix: Verilog truncates extra bits without warning. Ensure every wire connection has matching widths. Enable all warnings in Vivado simulator settings.

 addi treated as R-type

Symptom: `addi x1, x2, 5` adds `x3` instead of 5. Fix: opcode 0010011 (`addi`) uses `ALUSrc=1`, meaning ALU B comes from `ImmExt`, not `ReadData2`. Verify Control produces `ALUSrc=1` for this opcode.

A

RV32I Instruction Set Reference

All base integer instructions

Instruction	Format	Opcode	funct3	funct7	Operation
add rd, rs1, rs2	R	0110011	000	0000000	$rd = rs1 + rs2$
sub rd, rs1, rs2	R	0110011	000	0100000	$rd = rs1 - rs2$
and rd, rs1, rs2	R	0110011	111	0000000	$rd = rs1 \& rs2$
or rd, rs1, rs2	R	0110011	110	0000000	$rd = rs1 rs2$
xor rd, rs1, rs2	R	0110011	100	0000000	$rd = rs1 \wedge rs2$
slt rd, rs1, rs2	R	0110011	010	0000000	$rd = (rs1 < rs2) ? 1 : 0$ (signed)
sll rd, rs1, rs2	R	0110011	001	0000000	$rd = rs1 \ll rs2[4:0]$
srl rd, rs1, rs2	R	0110011	101	0000000	$rd = rs1 \gg rs2[4:0]$ (logical)
addi rd, rs1, imm	I	0010011	000	—	$rd = rs1 + \text{SignExt}(imm)$
andi rd, rs1, imm	I	0010011	111	—	$rd = rs1 \& \text{SignExt}(imm)$
ori rd, rs1, imm	I	0010011	110	—	$rd = rs1 \text{SignExt}(imm)$
slti rd, rs1, imm	I	0010011	010	—	$rd = (rs1 < \text{SignExt}(imm))$ $? 1 : 0$
lw rd, imm(rs1)	I	0000011	010	—	$rd = \text{Memory}[rs1 + \text{SignExt}(imm)]$
lh rd, imm(rs1)	I	0000011	001	—	$rd = \text{SignExt}(\text{Memory16}[rs1+imm])$
lb rd, imm(rs1)	I	0000011	000	—	$rd = \text{SignExt}(\text{Memory8}[rs1+imm])$
jalr rd, rs1, imm	I	1100111	000	—	$rd = PC+4; PC = (rs1+imm) \& \sim 1$
sw rs2, imm(rs1)	S	0100011	010	—	$\text{Memory}[rs1+imm] = rs2$
sh rs2, imm(rs1)	S	0100011	001	—	$\text{Memory16}[rs1+imm] = rs2[15:0]$
sb rs2, imm(rs1)	S	0100011	000	—	$\text{Memory8}[rs1+imm] = rs2[7:0]$
beq rs1,rs2,label	B	1100011	000	—	if $(rs1 == rs2)$ $PC = PC + \text{SignExt}(imm)$
bne rs1,rs2,label	B	1100011	001	—	if $(rs1 \neq rs2)$ $PC = PC + \text{SignExt}(imm)$
blt rs1,rs2,label	B	1100011	100	—	if $(rs1 < rs2)$ $PC = PC + \text{SignExt}(imm)$
bge rs1,rs2,label	B	1100011	101	—	if $(rs1 \geq rs2)$ $PC = PC + \text{SignExt}(imm)$
lui rd, imm	U	0110111	—	—	$rd = imm[31:12] \ll 12$
auipc rd, imm	U	0010111	—	—	$rd = PC + (imm[31:12] \ll 12)$
jal rd, label	J	1101111	—	—	$rd = PC+4; PC = PC + \text{SignExt}(imm)$

B**RISC-V Register Reference***ABI names and conventions*

Register	ABI Name	Calling Convention Role	Preserved Across Call?
x0	zero	Hardwired zero — always 0, writes ignored	N/A
x1	ra	Return address — where to return after a function call	No (caller saves)
x2	sp	Stack pointer — points to top of stack	Yes (callee saves)
x3	gp	Global pointer	No
x4	tp	Thread pointer	No
x5–x7	t0–t2	Temporary registers — scratch space in a function	No (caller saves)
x8	s0/fp	Saved register / frame pointer	Yes (callee saves)
x9	s1	Saved register	Yes (callee saves)
x10–x11	a0–a1	Function arguments and return values	No
x12–x17	a2–a7	Function arguments (remaining)	No
x18–x27	s2–s11	Saved registers — preserved across function calls	Yes (callee saves)
x28–x31	t3–t6	More temporaries	No (caller saves)

Build. Break. Debug. Repeat.*Every hardware engineer has wasted hours on a 1-character bug. You will too. That is how it works.*

Found an error? Have a question? Reach out — this guide was written for you.